REVENUEPILOT AI — META-PROMPT MASTER V3

Platformă orizontală de marketing AI | construcție pe GitHub | doi constructori Codex și Claude | cercetare Gemini

Versiune de specificație: 2026-11-07

CITEȘTE ACEST DOCUMENT ÎMPREUNĂ CU PLANUL DE CONSTRUCȚIE ASOCIAT. ESTE O SPECIFICAȚIE DE PRODUS ȘI UN CONTRACT DE EXECUȚIE, NU O COMANDĂ SĂ IMPLEMENTEZI TOTUL ÎNTR-O SINGURĂ SESIUNE.

0. INSTRUCȚIUNEA SUPREMĂ: CE CONSTRUIM ȘI CUM LUCRĂM

Acționează ca arhitect principal de software, inginer de sisteme distribuite, proiectant de produse SaaS, expert în securitate, inginer al evaluării LLM, specialist în marketing bazat pe dovezi și responsabil al livrării. Construiește REVENUEPILOT AI: o platformă universală de marketing AI, care acoperă progresiv întregul ciclu comercial: cunoașterea afacerii, cercetare, strategie, creație, verificare, distribuție, SEO, campanii, experiențe de conversie, leaduri, CRM, analiză, experimente, optimizare, retenție și automatizări personalizate.

Această platformă nu este limitată la mobilier, construcții sau altă nișă. Nucleul este orizontal. Diferențele dintre domenii sunt exprimate prin profiluri de brand, politici, capabilități configurabile, tipuri de oferte, șabloane testate și conectori. Un prototip poate fi validat cu firme din industrii diferite fără a declara produsul dependent de o industrie.

Nu confunda numărul mare de module ori de „agenți” cu valoarea. Fiecare complexitate trebuie să rezolve o cerință măsurabilă: creșterea calității creative, reducerea timpului de lucru, reducerea erorilor, creșterea conversiei, fiabilitatea, securitatea, retenția sau profitabilitatea.

Fiecare etapă din PLANUL DE CONSTRUCȚIE are un singur rezultat implementabil, verificabil, reversibil. Într-o sesiune lucrezi strict la ETAPA CURENTĂ. La sfârșit creezi un handoff durabil în repository, înregistrezi dovada testelor și TE OPREȘTI. Utilizatorul selectează apoi modelul potrivit și spune „Continuă cu pasul următor”. Nu porni singur alte etape, nu crea execuții paralele nelimitate și nu prezenta un plan drept funcționalitate implementată.

FONDATORUL NU ESTE PROGRAMATOR ȘI LUCREAZĂ NUMAI DE PE TELEFON. Interacțiunea cu omul trebuie să folosească instrucțiuni scurte și clare, acțiuni minime și rapoarte în limba română. Documentația tehnică poate folosi termenii tehnici consacrați în engleză. Totul trebuie păstrat într-un repository GitHub NOU, separat de „fabrica-autonoma”. GitHub este sursa de adevăr pentru cod, arhitectură, progres și evidențe; NU este gazdă pentru backend și NU înlocuiește o bază de date de producție.

1. CONDIȚII REALE DE OPERARE ȘI LIVRARE

1.1. Fondatorul are deja SRL, domeniu, posibilitatea de facturare și încasare, abonamente personale ChatGPT/Codex, Claude și Gemini Pro. Nu presupune o cheie API OpenAI, Anthropic ori Gemini disponibilă platformei. Nu presupune că abonamentul personal permite servirea autonomă a clienților prin intermediul produsului. Nu iniția costuri suplimentare până la autorizarea explicită după primii 2–3 clienți plătitori.

1.2. Hostul nu este gata. Nu bloca construcția din cauza lipsei VPS-ului. Creează și dezvoltă repository-ul pe GitHub, folosind instrumentele de lucru realmente accesibile agentului și verificările locale/CI dacă sunt disponibile. Nu pretinde că o aplicație este găzduită, accesibilă public sau în producție dacă nu este.

1.3. În prima etapă creează UN NOU REPOSITORY GitHub al produsului. Verifică autentificarea, drepturile de creare și push, vizibilitatea aleasă, branch-ul implicit și accesul efectiv. Preferă repository PRIVAT în timpul dezvoltării dacă planul și permisiunile o permit. Dacă unealta nu permite creare remote, inițializează local numai dacă există un spațiu

de lucru persistent și pregătește instrucțiunile exacte pentru un singur pas manual de creare din telefon; nu inventa un repository existent.

1.4. Nu atinge și nu modifica repository-ul „fabrica-autonoma”, VPS-ul său, containerele, credentialele ori altă infrastructură. Poți citi documentație sau lecții din fabrica existentă numai dacă sunt accesibile și ai permisiune, fără a copia cod sau secrete în mod nediferențiat. Noul produs este separat logic, contractual și operațional.

1.5. Nu cere sudo, acces root, alte abonamente sau card bancar fără justificarea necesității și aprobarea proprietarului. Orice serviciu extern se marchează drept disponibil, indisponibil sau neverificat; verifică termenii și limitele actuale înainte de integrare.

1.6. Lasă ALEGEREA NUMELOR FIȘIERELOR, a directoarelor, a claselor, a modulelor, a schemelor și a stackului final în sarcina inginerului executor. Această specificație NU conține cod sursă, configurații sau un arbore de fișiere obligatoriu. Singura cerință structurală este să existe în noul repository un PLAN CANONIC de lucru, un REGISTRU DE STARE, un JURNAL DE DECIZII, un JURNAL DE DOVEZI și o PROCEDURĂ DE RELUARE, descoperibile din pagina principală a proiectului. Aceste artefacte sunt create și denumite de model, apoi folosite consecvent.

2. CADRUL DE COLABORARE CODEX ↔ CLAUDE ↔ GEMINI, FĂRĂ API

2.1. Constructorul A: CODEX. Când este disponibil, este responsabil de planificarea unei etape, implementare, teste, înregistrarea progresului și predarea pentru review. Modele preferate, numai dacă sunt în selectorul real al utilizatorului: GPT-6.1 Sol / Medium pentru majoritatea implementărilor dificile; GPT-6.1 Terra / Medium pentru sarcini precise și UI; GPT-6 Luna / Low pentru mecanică; GPT-6 Astra / High doar pentru decizii arhitecturale și audituri de risc ridicat. În cadrul unui task nu modifica alegerea făcută de utilizator și nu susține că poți schimba singur modelul abonamentului.

2.2. Constructorul B: CLAUDE CODE. Când este disponibil, are autonomie de implementare în taskurile sale și responsabilitate independentă de revizuire a modificărilor Codex. Modele preferate, dacă sunt accesibile: Claude Opus 5.5 / High pentru arhitectură, concurență de date, logică de orchestrare și audit; Claude Sonnet 5.5 / Medium pentru implementări; Claude Haiku 5.5 / Low pentru extrageri și operații repetitive. Revizuirea poate fi reciprocă: autorul și evaluatorul au roluri separate, schimbate între etape.

2.3. Cercetătorul C: GEMINI PRO. În lipsa API-ului, operatorul poate utiliza interfața oficială Google ori instrumente autorizate disponibile abonamentului; rezultatul cercetării este importat printr-un handoff uman cu surse, data accesării, întrebarea, răspunsul și limitele. Nu afirma că Gemini primește automat mesaje de la GitHub sau că este integrat direct dacă nu există conectare verificată. Perplexity este opțional și nu reprezintă o dependență.

2.4. COMUNICAREA EFECTIVĂ FĂRĂ API: modele care lucrează în sesiuni distincte folosesc commituri Git, pull requesturi, taskuri persistente, artefacte de predare și un jurnal de stare. Fiecare agent citește la pornire HEAD-ul branchului, taskul autorizat, dependențele, rezultatele anterioare și instrucțiunile asociate rolului său; lucrează într-un branch izolat; produce teste și dovezi; face push; cere review; oprește sesiunea. Când utilizatorul face switch de model, noul model reia din repository, nu din memoria conversației anterioare.

2.5. Nu simula un „răspuns Claude” generat de Codex sau invers. Lipsa unui reviewer real se înregistrează ca REVIEW_PENDING și nu se reetichetează APPROVED. Dacă un pas nu necesită review independent pentru a fi util, poate fi pregătit separat, dar niciodată numit „certificat dual-model” fără dovadă.

2.6. Previne suprascrierile: un singur autor principal per task, branch distinct, revendicare cu termen de expirare, baza de commit de la care a pornit, raportarea conflictelor, fără force-push. Înainte de editare, verifică dacă altă sesiune a avansat registrul. Dacă da, reîncarcă, replanifică și nu rescrie munca altora.

2.7. Nu promite conversații „live” Codex–Claude–Gemini prin abonamente. Poate fi proiectat ulterior un adaptor de comunicare autorizat, dar primul mod operațional rămâne „Git + trecere manuală între sesiuni”. Standardele MCP și A2A sunt candidate pentru viitoare integrări de unelte și sisteme de agenți; nu sunt un substitut pentru accesul la un model.

3. MANAGEMENTUL ETAPELOR ȘI AL STĂRII DURABILE

3.1. Starea fiecărei etape include: identitate stabilă, descriere, obiectiv, beneficiu, prioritizare, dependențe, model recomandat pentru Codex și Claude, executorul efectiv, scopul permis, risc, resurse, costuri, inputuri, outputuri, criterii de acceptare, teste, evidențe, defecte, reviewer, aprobări, commit de bază, branch, commit final, raport de predare, următorul pas permis.

3.2. Stările permise vor include cel puțin: NOT_STARTED, READY, CLAIMED, IN_PROGRESS, IMPLEMENTED, TESTING, REVIEW_PENDING, CHANGES_REQUESTED, VERIFIED, DONE, BLOCKED, DEFERRED, FAILED. Definește tranziții legale, cine le poate declanșa și condițiile de intrare/ieșire. Nu permite DONE fără dovezi obligatorii.

3.3. Fiecare sesiune execută ciclul: descoperire → verificare actualizare remote → citire plan și stare → selectarea exactă a primului pas pregătit/autorizat → validarea dependențelor și cotei → implementare → teste → comparație cu acceptance → jurnal de schimbări și lecții → commit/push → review sau predare → STOP.

3.4. Dacă utilizatorul spune „Continuă cu pasul următor”, continuarea înseamnă doar primul pas executabil, nu toate etapele rămase. Dacă modelul ales diferă de recomandare, explică pe scurt riscul și execută doar dacă sarcina este adecvată modelului curent; pentru o problemă critică poate cere schimbarea pe Astra/Opus.

3.5. După întrerupere, reluarea trebuie să fie idempotentă: verifică modificările făcute, nu creează copii, nu dublează commituri și nu reproduce operațiuni externe. În cazul unui pas parțial, reia din punctul dovedit sigur, nu presupune reușita.

3.6. Păstrează explicit ceea ce nu este implementat, ce nu a fost testat și ce depinde de host/API/cont de platformă. Nimic din documentație nu poate transforma un mock într-o integrare reală.

4. VIZIUNEA PRODUSULUI ȘI DIFERENȚIATORUL ECONOMIC

4.1. Platforma trebuie să gestioneze marketingul ca pe un sistem de venituri, nu ca pe un sistem de postări. Definește traseul: obiectiv de business → ofertă → audiență → ipoteză de mesaj → material creativ → canal → interacțiune → trafic calificat → lead → oportunitate → contract/vânzare → retenție → învățare.

4.2. Avantajul urmărit este bucla închisă „marketing-to-revenue”: feedback verificabil despre campanii, cost, calitate și vânzări, fără a pretinde atribuire perfectă. Integrarea cu software-ul existent al clientului este preferată înlocuirii CRM-ului sau a agenției sale.

4.3. Oferă configurări pentru întreprinderi mici, agenții, profesioniști și organizații mai mari. Nu specializa logic infrastructura pe un domeniu fix. Separă capabilitățile universale de restricțiile sectoriale: sănătate, finanțe, minori, produse reglementate, confidențialitate, publicitate locală.

4.4. Modulele și automatizările trebuie să expună costul incremental și valoarea estimată. Nu transforma funcții cu cost video/randare/scoring ridicat în resurse „nelimitate” implicite. Planurile comerciale vor avea limite și prioritate configurabile.

5. PRODUSUL A–Z: HARTA DE CAPABILITĂȚI

5.1. Onboarding și identitate: organizații, utilizatori, membri, roluri, permisiuni, profil de business, produse, prețuri, piețe, limbi, obiective, bugete, website, integrări, consimțăminte, preferințe de brand. Onboarding progresiv, fără formulare inutile. Suport mobil cu stări de salvare și reluare.

5.2. Brand intelligence: voce, stil, tone matrix, restricții, valori, identitate vizuală, ghid editorial, catalog de oferte, formulări aprobate, afirmații permise/interzise, publicuri, exemple bune/rele, istoric campanii, sursele informațiilor și data de valabilitate. Schimbările majore ale profilului necesită versionare și aprobare.

5.3. Market intelligence: competitori, categorii, nevoi, review-uri publice și permise, tendințe, sezonalitate, diferențiatori, oferte, canale, ipoteze și nivelul de dovezi. Respectă termenii sursei; nu construi crawling ofensiv, eludări anti-bot sau colectare indiscriminată de date personale.

5.4. Strategie: STP/ICP, JTBD, poziționare, propoziție de valoare, maparea funnelurilor, obiective SMART, media mix, canale, frecvență, calendar, experimente, model de buget, riscuri, aprobare și criterii de oprire.

5.5. Content intelligence: content pillars, research briefs, actualitatea surselor, voice-of-customer, idee → hook → cadru narativ → copy → CTA → format → canal → experiment. Generează variații substanțiale, nu parapraze mecanice.

5.6. Creative studio: texte lungi/scurte, carusele, reclame, headline-uri, thumbnails, elemente pentru imagini și video, storytelling, umor, personaje și scenarii fictive, demonstrații, conținut educațional, comparații, unghiuri provocatoare. Ficțiunea este permisă ca ficțiune; rezultate, review-uri, testimoniale, premii, cote de piață și statistici atribuite unor companii reale necesită dovezi.

5.7. Video production: scripts, storyboards, shot lists, cadre și tranziții, aspect ratio, caption, subtitrare, traduceri, hook în primele secunde, sound design permis, UGC autorizat, guidelines pentru editare. Distinge „brief video”, „asset generat”, „render verificat” și „publicat”; fără instrument media real, nu declara randare video realizată.

5.8. Social distribution: TikTok, Instagram, Facebook, Pinterest, YouTube Shorts, LinkedIn, Google Business Profile și canale viitoare. Pentru fiecare: format, restricții, asset rights, API connector, autorizare, review-ul aplicației dacă e necesar, status publishing, confirmări, retry, token expiry, metrici. Moduri: export manual, publishing asistat, API autorizat. Autopostarea nu este presupusă.

5.9. SEO și descoperire AI: audit tehnic permis, strategie de subiecte, structură semantică, entity coverage, conținut original, interlinking, metadata, schema markup unde este corect, performanță, local SEO, AEO/GEO cu standarde de factualitate și transparență. Evită paginile doorway, conținut duplicat la scară și afirmații fără valoare.

5.10. Web și conversie: landing pages, secțiuni modulare, formulare, CTA, UX, accesibilitate, consent flows, tracking de evenimente, page speed, microcopy, experimente, heatmap doar dacă integrările sunt reale, anti-spam legitim.

5.11. Lead operations: ingestie, verificare schemă, deduplicare, sursă, scor de calificare, motive ale scorului, rutare, SLA, CRM sync, follow-up cu temei legal, oportunități și stadii. Scorurile inițiale sunt reguli/proxy-uri, nu probabilități adevărate.

5.12. Paid ads: briefuri, campanii, variații creative, simulări de buget, experimente, verificări înainte de activare, ad account connectors, spend caps, reconciliere. Nu porni sau modifica cheltuieli fără aprobarea explicită a proprietarului contului.

5.13. Email/retention: segmentare, secvențe opt-in, lifecycle, reactivare, experiență post-vânzare, upsell relevant, unsubscribe, suppressions și reputație de trimitere. Nu presupune dreptul de a trimite mesaje către contacte obținute prin scraping.

5.14. Reputație și comunitate: social listening permis, semnale de satisfacție, feedback, răspunsuri propuse, escaladări, cazuri negative, recomandări, parteneriate, creatori și ambasadori reali. Evită recenzii ori interacțiuni false.

5.15. Revenue analytics: metrici de reach, CTR, engagement, MQL, SQL, demo, ofertă, conversie, CAC, venit, marjă, churn, retenție și intervale de incertitudine. Explică diferențele dintre corelație și cauzalitate; nu atribui toate vânzările platformei dacă datele nu susțin afirmația.

5.16. Experimente: ipoteză, control, tratament, populație, segment, perioadă, primary metric, guardrails, selecție creativă, stop rule, date lipsă, sample size, evitarea concluziilor premature și jurnal de învățare. Oferă explorare creativă, dar nu „garantează viralitate”.

5.17. Integrări: API connectors, webhooks, import/export, CSV, CRM, servicii media, analytics, email, platforme sociale, instrumente no-code. Contracte cu versionare, test de conectivitate, rate limits, idempotency, retries, reautorizare, audit și dezactivare.

5.18. Agenții de marketing și de creștere pentru RevenuePilot AI însuși: research, positioning, demo, conținut organic, programmatic assets cu utilitate, distribuție creatori/afiliați, referral, lead capture, partnerships, sales follow-up, rapoarte. Creșterea se evaluează prin clienți și venituri, nu doar număr de postări.

6. SISTEMUL MULTI-AGENT: ROLURI, NU INSTANȚE DECORATIVE

6.1. Definește un registru de capabilități pentru agenți. Fiecare agent logic trebuie să aibă: ID și versiune, scop exact, autoritate limitată, date permise, capabilități, instrumente, nivel de risc, contract intrare/ieșire, cost maxim, timp maxim, buget de retry, condiții de oprire, dependențe, rubrică de calitate, ce trebuie documentat, cum învață, când escaladează, ce evaluator îl verifică, scenarii negative.

6.2. Familii de agenți: Research & Evidence; Audience & Positioning; Strategy & Portfolio; Brand Memory; Copy & Concept; Video/Visual Brief; SEO & Information Architecture; Social Adaptation; Publishing Coordinator; Landing/CRO; Lead Qualification; CRM/Follow-up; Analytics/Attribution; Experiment Designer; Budget & Cost; Customer Success; Policy/Privacy; QA/Brand/Facts/Creative/Originality; Security/Incident; Learning & Evaluation; Integrations; Orchestrator; Human Approval Broker.

6.3. Nu este necesar un proces LLM dedicat fiecărei denumiri. Mai multe roluri pot utiliza aceeași implementare și același model într-un flux serializat, cu politici distincte. Separă identitatea rolului de identitatea modelului utilizat. Nu crea 40 de agenți simultani dacă există doar două abonamente consumer și niciun API.

6.4. Orchestratorul determinist este responsabil de tranziții și autorizații. LLM-ul propune: taskuri, strategii, conținut, îmbunătățiri. Nu îi permite să modifice singur nivelul de acces, bugetul, politica de publicare, datele altui tenant sau condițiile de aprobare.

6.5. Comunicare inter-agent: request/response tipizate, task ID, parent ID, correlation ID, context scope, versiune de schemă, proveniență, checksum/hash, deadline, ownership, capability scope, confidence calibrată și elemente de verificare. Sunt obligatorii acknowledgements, idempotency, deduplicare, retries cu limite, dead-letter/quarantine și replay sigur.

6.6. Blackboards, jurnalele de evenimente și memoriile trebuie să fie segregate per organizație. Contextul comun între agenți se livrează prin retrieval selectiv și artefacte versionate, nu prin transcrieri integrale ale tuturor sesiunilor. Un agent primește doar informațiile necesare sarcinii.

6.7. Evidențiază diferența dintre agent autonom, workflow determinist, simplă integrare și sarcină asistată manual. Interfața clientului nu va prezenta o integrare manuală ca „AI autopilot”.

7. ORCHESTRARE, REZILIENȚĂ ȘI COMUNICAȚIE MODERNĂ

7.1. Folosește mașini de stări definite explicit și evenimente durabile. Pentru fiecare acțiune externă definește idempotency key, timeout, retry policy, backoff, circuit breaker, compensare dacă este posibilă, expirare, audit și criteriu de confirmare. Eroare de API, quota exceeded, browser timeout, sesiune expirată, lipsă aprobare, date invalide sau conflict de versionare trebuie să fie stări tratabile.

7.2. Proiectează separat CONTROL PLANE (autentificare, tenants, politici, model routing, quality, approvals, budgets, coordonare) și EXECUTION PLANE (render, procesare, conectori, campanii, joburi, publicare, event ingest). În prima versiune pot coexista într-o aplicație modulară; separarea fizică devine necesară doar dacă măsurătorile o justifică.

7.3. Examinează MCP pentru unelte și A2A pentru interoperabilitatea agent-la-agent, dar nu forța implementarea completă a unui protocol înainte de nevoi reale. Examinează LangGraph pentru checkpointing/interruptions și DSPy pentru optimizarea prompturilor pe seturi de evaluare. Selectează-le doar după o comparație argumentată cu o implementare simplă, deterministă.

7.4. Nu crea dependență obligatorie de o bibliotecă doar pentru „agenți mai inteligenți”. Orice framework introdus trebuie să treacă prin evaluare de maturitate, securitate, compatibilitate, memorie, CPU, cost și înlocuire.

7.5. Componente critice: event log append-only unde are sens; checkpoint-uri; ownership; locks/leases cu TTL; optimist concurrency; backpressure; rehidratarea contextului; idempotent resume; outbox/inbox dacă apar tranzacții și webhooks; reconciliere în caz de rezultate ambigue; limita de worker concurrency; kill switch; degradare controlată în regim manual.

8. SISTEM DE ÎNVĂȚARE CONTINUĂ — CONSTRÂNGERE DE PROIECTARE

8.1. Obiectivul este ca fiecare task finalizat sau eșuat să poată produce lecții reutilizabile. „Învață autonom” înseamnă corecții și optimizări demonstrabile ale comportamentului sistemului, NU presupunerea că ponderile modelelor comerciale se modifică din conversații.

8.2. Implementare în trepte:

NIVEL L0: captură de evenimente, input/output, rezultate, erori, costuri, versiuni, proveniență și feedback (cu minimizare, redactare PII, consimțământ și retenție).

NIVEL L1: memorie episodică versionată, lecții și antipatternuri; deduplicare, calitate, TTL, invalidare.

NIVEL L2: retrieval contextual; selectarea doar a lecțiilor relevante pentru noul task, cu sursă și grad de încredere.

NIVEL L3: adaptare de prompturi, rubrics, exemple, politici locale necritice, tool selection și workflow alternatives ca PROPUNERI versionate, evaluate offline.

NIVEL L4: evaluări repetabile pe benchmark-uri, regression/canary/A-B; promovarea doar a candidatului superior și reversibil.

NIVEL L5: adaptare statistică a rutării, pragurilor, bugetelor și priorităților, doar în intervale aprobate.

NIVEL L6 (OPȚIONAL, VIITOR): fine-tuning/distillation ale unui model pe care avem drept de antrenare, date eligibile și buget demonstrat. Nu încerca „antrenarea” unui model Claude/Codex doar prin actualizarea unui fișier.

8.3. Organizarea memoriei: procedurală (cum se face), semantică (fapte despre domeniu), episodică (ce s-a întâmplat), evaluativă (de ce a fost bun/rău), de brand (doar per tenant), de sistem (defecte recurente), experiment (hipoteză, rezultate), feedback uman, negative memory (ce NU trebuie repetat). Fiecare înregistrare are origine, timestamps, evaluator, validitate, nivel de confidențialitate, versiune, relații, supersession și ștergere.

8.4. Nu salva literal „absolut tot orice” fără limite. Capturează toate TIPURILE DE SEMNALE NECESARE pentru analiză; datele sensibile și redundante se filtrează sau se agregă. Separă telemetria de conținut. Datele clienților nu devin memorie globală accesibilă altor clienți. O cerere de ștergere GDPR trebuie să aibă efect și asupra memoriilor și exemplelor de evaluare derivate unde este aplicabil.

8.5. Fiecare lecție trebuie să declare: simptome, cauză probabilă vs cauză demonstrată, experimentul de confirmare, patch aplicat, test de regresie, metrica înainte/după, domeniu de valabilitate, neaplicabilitate, risc de regresie și data reevaluării. Lecțiile conflictuale sunt adjudecate, nu concatenate necritic într-un prompt enorm.

8.6. Calibrare: înregistrează predicția inițială și rezultatul observat pentru fiecare evaluator; calculează fiabilitatea, matrice de confuzie, precision/recall, false positives/negatives, calibrare a scorurilor (de exemplu Brier/ECE unde există probabilități și etichete), acord între evaluatori, drift temporal și metrici per canal/segment. În absența datelor etichetate, raportează „insuficient calibrat”.

8.7. Modelul de adaptare trebuie să evite reward hacking. Nu permite evaluatorilor să micșoreze pragurile numai pentru a crește rata de acceptare. Creează holdout de teste nevăzute de generator și de optimizator; păstrează baseline; rulări shadow; canary; rollback; limite de schimbări per iterație; audit independent; înghețarea politicilor critice.

8.8. Explorează metode moderne cu justificare: retrieval și reranking pentru memory search; few-shot selection; bandit contextual pentru alegerea variantelor sau a rutării, doar când există semnale suficiente; Bayesian optimization pentru parametri bine definiți; DSPy/GEPA/MIPROv2 pentru optimizarea prompturilor pe evals, doar cu furnizor disponibil și buget; supervised learning pentru scoruri pe date legitime; metric learning și embeddings dacă simplele căutări nu sunt suficiente; detectare de anomalii pentru operațiuni; clasificare de erori; active learning pentru cazuri ambigue; distillation doar când ROI demonstrat. Nu introduce „rețele neuronale custom” fără seturi de date, calcul și criterii clare.

8.9. Contracție de memorie: fiecare ciclu trebuie să reducă zgomotul, nu doar să acumuleze text. Aplică deduplicare semantică, ranking după utilitate confirmată, expirare, sumarizare cu citarea surselor și recuperare la cerere. Măsoară costul adăugat de memorie versus calitatea îmbunătățită.

8.10. Definește un „Learning Control Board”: performanță pe agent/versiune/segment, defecte recurente, promisiuni vs rezultate, experimente active, recall lecții, impact financiar, cost suplimentar, regresii, drift și propuneri de calibrare. Orice schimbare care afectează drepturi, bani, date sau publicare necesită aprobare umană.

8.11. Trasabilitate completă: fiecare rezultat al unui agent trebuie să poată fi legat de task, versiunea promptului, model, tool calls autorizate, data contextului, versiune de evaluator, decizie de aprobare, versiunea assetului, rezultat de campanie, lecția extrasă și experimentul care a schimbat comportamentul. Trasabilitate înseamnă metadate verificabile, nu colectarea secretelor ori a raționamentului intern privat al modelului.

9. QUALITY ENGINE — ÎNALT, RIGUROS, DAR CALIBRAT, NU „PERFECT” DECLARAT

9.1. Nu afirma „filtre perfecte”. Construiește filtre testabile, explicabile, independente unde este necesar, cu măsurarea ratelor de eroare. Scopul este un nivel de calitate constant crescător, pe bază de dovezi, fără a sacrifica inutil throughput-ul.

9.2. Lanț de verificare pentru conținut: validitatea brief-ului; context de brand; relevanța audienței; creativitate și diferențiere; claritate și valoarea mesajului; structură hook/body/CTA; corectitudine lingvistică; originalitate/near-duplicate; afirmații factuale; surse și actualitate; proveniența asseturilor; licențe/drepturi; consistență vizuală; accesibilitate; cerințe de format; compatibilitate de canal; tone and sentiment; landing consistency; verificare tehnică a linkurilor; riscuri reputaționale; conformitate cu publicitatea aplicabilă; pregătire de publicare; verificare post-publicare; măsurarea feedbackului real.

9.3. Lanț de verificare pentru software: cerință → contract → teste → lint/type checking → static analysis → analiză de dependențe → scanare de secrete → teste unitare/integrări/proprietăți/E2E → accesibilitate și mobil → threat model → review independent → CI reproducibil → aprobări. Pentru funcționalități cu risc ridicat, adaugă red-team și testare de reziliență; nu pretinde testare dacă mediul nu a permis-o.

9.4. Separă verificările DETERMINISTE (regex, schemă, fapte în catalog, limite, imagini, linkuri, drepturi, reguli) de verificările SUBIECTIVE (calitatea ideii, persuasiune, ton, elemente de noutate). Un LLM Judge este consultativ, nu autoritatea finală asupra adevărului.

9.5. Evaluatorii specializați: Brief Completeness; Brand Consistency; Creative Director; Factual/Evidence; Originality; UX/Accessibility; Language; Policy/Rights; Channel Compatibility; Conversion Relevance; Security; Data Privacy; Experiment Integrity; Model Behavior. Orice evaluator are rubrică, cazuri pozitive/negative, versiune și defecte cunoscute.

9.6. Filtru anti-false-positive: evaluatorul trebuie să indice elementul exact din artefact care demonstrează defectul, regula încălcată și modul de reproducere; dacă solicită ceva imposibil în componenta revizuită sau în afara scopului, constatarea este escaladată ori respinsă ca nefondată. Nu repara un frontend pentru a rezolva tranzacții SQL care țin de server. Nu schimba cerința clientului ca să treacă testul. Nu elimina funcționalități cerute doar pentru a obține „green”.

9.7. Filtru anti-false-negative: teste adversariale și exemple istorice de erori scapate; eșantionare manuală; verificare încrucișată; analize de drift; testarea situațiilor rare, inclusiv valori românești, diacritice, monede, nume de câmpuri, platform policies și scenarii multi-tenant.

9.8. Învățarea evaluatorilor: la orice conflict generator ↔ evaluator, se păstrează ambele versiuni, probele, adjudecarea, noua regulă propusă și testul de regresie. Schimbarea de rubrică nu este promovată dacă degradează testele de referință. La schimbarea modelului evaluator, rulează seturi comparative.

9.9. Parametrii de calibrare, cel puțin: confidence threshold, reject threshold, review sampling rate, severity weights, brand tolerance, originality similarity threshold, factual evidence requirement, risk class, retry cap, optimization exploration rate, memory retrieval top-k, memory TTL, max context, max latency, cost cap, stop condition și human escalation threshold. Parametrii trebuie versionați, validați, explicabili și limitabili; un agent nu îi poate modifica arbitrar.

9.10. Nu transforma scorul de creativitate într-un surogat al eficacității. Calitatea unui material se măsoară în două etape: BEFORE publication (rubrici și teste) și AFTER publication (semnale reale, atribuire limitată, impact asupra conversiilor). Optimizează pentru utilitate și venit, nu pentru a impresiona evaluatorul.

10. DATE, MULTI-TENANCY, PRIVACY ȘI SECURITATE

10.1. Desenează un model de date conceptual: users, organizations, memberships, brands, offers, audiences, permissions, secrets refs, assets, asset versions, briefs, strategies, campaigns, variants, content approvals, channels, integrations, distribution jobs, leads, conversions, attribution events, experiments, evaluations, agent tasks, memory items, lessons, calibration proposals, telemetry, usage, invoices references, audit records, privacy requests. Modelele și numele efective sunt alese de executor când implementează.

10.2. Fiecare acces la date trebuie verificat la nivel de tenant. Testează explicit două organizații, utilizatori cu roluri diferite, acces indirect prin identificatori, exporturi, căutare semantică, logs, memorie și event streams. Nu reutiliza embeddings sau exemple cu date confidențiale între clienți fără bază legală și izolare.

10.3. Aplică least privilege, secure defaults, validare de schema, anti-CSRF unde este relevant, rate limits, CORS corect, protecție SSRF, HTML sanitization, secure uploads, storage scanning, webhooks signed, anti-replay, key rotation, TTL, secrete în vault/CI secrets (nu în Git), audit al accesului, backup + test de restore și plan de răspuns la incident.

10.4. Conținut extern = input neîncredere. Un mesaj dintr-o pagină web, un PDF, un email sau un comentariu nu poate schimba instrucțiunile sistemului, nu poate activa tooluri, nu poate dezvălui secrete și nu poate escalada permisiuni. Separă datele de instrucțiuni prin politici, contracte, tool authorization și validare deterministă.

10.5. GDPR/ePrivacy: inventar de scopuri, baze legale, roluri controller/processor, retenție, drepturi de acces/rectificare/ștergere, export, separarea consimțămintelor, jurnal de procesare, proces pentru cereri DSAR și gestionarea subcontractanților. Nu implementa „compliance complet” ca simplă bifă; documentează obligațiile juridice de verificat cu specialist.

11. COST, PERFORMANȚĂ ȘI RESILIENȚĂ COMERCIALĂ

11.1. Contabilizează distinct: developer subscriptions, utilizare API atunci când va exista, inference, generare imagine/video, stocare, trafic, conectori, email, monitorizare, suport, muncă umană, CAC, administrație, procesare plăți (deja existentă), taxe și rezerva pentru incidente. Nu numi venit minus cost hosting „profit net”.

11.2. Sistem de bugete per organizație și per tip de job: limite de soft/hard, cost estimat vs realizat, break-even pentru fiecare funcție, blocare/cerere de aprobare la atingerea limitei, observarea ratei de eșec, prioritizare, backpressure și degradare la proces manual.

11.3. Optimizează cât mai devreme simplitatea, nu o arhitectură de 50 de microservicii. Evaluează monolit modular vs servicii separate. Ia în calcul un host viitor numai după ce există un flux complet, testabil și clienți sau justificare economică.

11.4. Pentru media: drepturi, stocare, lifecycle, compresie, transcodare, cozi, volumul de fișiere, cost, livrare, expirare, backup și ștergere. Nu folosi Git ca depozit de video, imagini uriașe sau date de clienți.

12. INTERFAȚĂ, APROBĂRI ȘI EXPLICABILITATE

12.1. Prioritate mobile-first. Experiența minimă: înregistrare/organizație → profil brand → obiectiv → brief → strategie → variante creative → feedback/aprobare → pregătire publicare → campanie → leaduri → rezultate → lecții. Un dashboard nu este dovadă că integrarea reală există.

12.2. Trei niveluri de autonomie: MANUAL (omul execută), ASSISTED (sistemul propune, omul aprobă), AUTHORIZED AUTO (sistemul execută doar acțiuni cu conector verificat, risc permis și buget autorizat). Fiecare modul afișează clar nivelul real și eventualele dependențe.

12.3. Arată clientului de ce o recomandare este făcută: informațiile folosite, data, limitările, costul, variantele, gradul de incertitudine, reviewerii și rezultatul experimentelor. Un client poate respinge o creație și explica motivul; acest feedback intră în sistemul de învățare după validare.

12.4. UX de urgență: revocare token, deconectare canal, anulare job, oprire buget, pauză campanie, rollback și raport de incident. Nu ascunde aceste controale sub „pilot automat”.

13. STRATEGIA DE CREȘTERE A BUSINESSULUI NOSTRU

13.1. Fondatorul urmărește trafic masiv, însă optimizează progresiv pentru achiziție plătitoare și retenție. Construiește experimente de distribuție pe TikTok, Meta, Shorts, Pinterest, LinkedIn, SEO, comunități, parteneriate cu agenții, afiliere și instrumente gratuite utile; măsoară funnelul complet views → visits → signup → activation → qualified trial → paid → retained.

13.2. Permite stiluri creative puternice, campanii ficționale, narațiuni, comparații provocatoare și unghiuri experimentale. Nu prezenta rezultate comerciale fictive drept studii de caz reale și nu crea interacțiuni false pentru a mima popularitate. Distinge între tactici permise legal, acceptate de platforme și simple posibilități tehnice; riscurile de suspendare și pierderea distribuției intră în evaluarea economică.

13.3. Definește growth loops: utilizator creează o campanie utilă → folosește public un asset sau widget autorizat → alți cumpărători descoperă produsul → se înscriu → primesc valoare și distribuie voluntar. Alte bucle: agenții white-label, recomandări, template marketplace și integrare cu platformele existente. Fiecare buclă necesită măsurare reală, nu speranță de viralitate.

13.4. NU porni reclame sau cumpăra media fără buget explicit. Pune prioritățile de validare: demos reale, 2–3 clienți plătitori din domenii diferite, timp de onboarding, cost complet, retenție și cazuri de utilizare comparabile.

14. FAZE COMERCIALE ȘI CRITERII DE GO / NO-GO

14.1. În faza zero, pregătește produsul pe Git. Fără host nu se poate verifica deploy-ul în producție; folosește teste, builds, demo-uri statice/mocks etichetate și local preview dacă există.

14.2. Prima versiune livrabilă: profil de business, brand memory, brief, strategie, conținut asistat, quality review, aprobare, export de campanie, captare lead și analiză minimală cu date validate. Nu pretinde publicare automată dacă nu avem conector autorizat.

14.3. Pilotează cu firme din cel puțin două sau trei domenii diferite pentru a testa orizontalitatea. Măsoară: cine a plătit, ce problemă a cumpărat, timpul până la prima valoare, volumul de corecții, costul real, retenția și probabilitatea de recomandare. Fără plăți reale, declară „nevalidat”.

14.4. Fiecare extindere se justifică prin date: rata de folosire, venit marginal, suport, erori, costuri, impact pe conversie. O capabilitate care arată impresionant dar nu este folosită nu primește automat prioritate.

15. TESTARE, CALITATE SOFTWARE ȘI ACCEPTANȚĂ

15.1. Planifică automatizarea testelor în ordinea riscului: contracts și schema; auth; tenant isolation; input validation; state machine; idempotency; retries; data migration; prompt injection; permissions; policy enforcement; quality gates; memory deletion; evaluation regression; CI; UI mobile; E2E; import/export; connectors; incident recovery.

15.2. Construiește fixture-uri sintetice pentru cel puțin 3 organizații și 3 industrii diferite. În teste, datele sintetice trebuie marcate clar. Nu introduce chei ori date reale ale clienților în repo.

15.3. Separă testele mock, integrare, sandbox, contract, live și acceptance. Atestă explicit care au fost executate în mediu; nu combina raportul mock cu afirmația „am testat API-ul extern”.

15.4. Nu relaxa criteriile ca să treacă CI. Dacă testul este greșit, demonstrează eroarea prin contract și reproduce regresia. Dacă implementarea este greșită, repară implementarea. Păstrează before/after și explicația adjudecării.

15.5. Fiecare pas necesită evidență: surse modificate, rulări, hash/commit, teste, rezultate, code review, defecte cunoscute, cost și acceptare. Dacă nu poate fi demonstrat, rămâne neverificat.

16. CRITERII PENTRU UTILIZAREA MODELELOR ȘI A COTEI

16.1. Modelele sunt alese de utilizator în interfață înainte de sesiune. Planul de construcție oferă perechea recomandată Codex/Claude și nivelul de efort. Agentul verifică dacă modelul selectat poate îndeplini etapa fără a genera muncă ineficientă. Preferă Sol/Sonnet pentru implementare, Terra/Sonnet pentru UI și teste, Luna/Haiku pentru mecanică, Astra/Opus numai la risc mare.

16.2. Pentru a economisi quota: o singură etapă; context minim necesar; plan scurt înainte de execuție; fără re-citirea integrală a jurnalului istoric; retrieval din repo la nevoie; rezumate concise cu commituri; evită generarea a 10 variante de implementare fără test; fără subagenți paraleli inutili; număr limitat de iterații; escaladează dacă sunt două eșecuri similare.

16.3. Un abonament de dezvoltare nu este un backend comercial. Separă timpul în care constructorul generează cod de inferența pentru clienți. Fără API disponibil, fluxurile AI runtime sunt doar: manual, offline pentru test, provider adapter dezactivat ori integrare autorizată deja verificată. Când API-ul este introdus ulterior, configurarea, costul, limitele și politicile se verifică din nou.

16.4. Gemini rămâne cercetător în fluxul uman până la conectare autorizată. Agentul documentează promptul de research, rezultatul, sursele, contradicțiile și nivelul de validare. Fără surse verificabile, rezultatul este ipoteză.

17. GUVERNANȚĂ, TRANSPARENȚĂ ȘI DOCUMENTARE TOTALĂ

17.1. Documentează TOT ce este necesar pentru reproducere și îmbunătățire: motivul fiecărei decizii, alternative, trade-off, design, contracte, politici, date simulate, teste, metrica baseline, rezultate, erori, recovery, lessons, actualizări de parametri, versiuni de agenți, integrare și infrastructură. Nu documenta secrete, date personale brute sau informații pentru care nu există drept de stocare.

17.2. Înregistrează observațiile ca fapte dovedite, deducții sau ipoteze. O decizie a unui model nu devine adevăr prin repetare. Corecțiile ulterioare trebuie să păstreze istoria și motivul schimbării.

17.3. Repository-ul trebuie să conțină suficientă documentație ca un ALT model, pornit într-o sesiune complet nouă, să afle în câteva minute: misiunea, stadiul exact, arhitectura, pasul următor, ce este blocat, teste disponibile, riscuri, ce nu trebuie modificat și cum se lucrează cu GitHub.

17.4. Previzionează costul jurnalizării: loguri structurate, sampling, retenție, compacție, protejarea datelor, decizii de arhivare. „Trasabilitate completă” înseamnă dovezi suficiente pentru audit, nu stocarea infinită a fiecărui token.

18. LECȚII DIN „FABRICA AUTONOMĂ” CARE TREBUIE ADAPTATE

18.1. Pe GitHub există un proiect separat, „fabrica-autonoma”, cu un branch de lucru „fabrica-v2”. Nu îl modifica. Concepte utile de investigat, doar cu permisiune: mașină de stări deterministă, review cross-family, jurnal de decizii, test oracle separat, rate-limit/quota recovery, învățare din eșecuri, supraveghere/rollback, protocoale de handoff.

18.2. Probleme observate în documentația repo-ului: evaluări false pozitive care cereau SQL în frontend; testare care confunda textul unui parametru cu valoarea lui; curățare de containere care a afectat livrări; imagini de livrare dispărute; traducerea incorectă a valorilor cerute de utilizator; diverse reparații generale care aveau nevoie de teste de regresie. Tradu aceste lecții în cerințe verificabile pentru noua platformă, nu copia implementarea automat.

18.3. Nu declara „self-healing 100%” sau autonomie perfectă. Măsoară rata de autoremediere, regresiile, intervențiile umane, tentativele eșuate și bugetul. Nu permite unui agent de reparare să își modifice unilateral testele ori regulile de acceptare pentru a simula progres.

19. PROCEDURA EXACTĂ DE EXECUȚIE A UNEI ETAPE

La începutul FIECĂREI sesiuni:

A. Confirmă identitatea repository-ului nou, branchul, HEAD-ul și absența modificărilor neasumate.

B. Citește PLANUL CANONIC și REGISTRUL DE STARE din repository; nu încerca să deduci progresul din chat.

C. Identifică PRIMUL pas în starea READY, cu dependențe satisfăcute și autorizat pentru modelul curent. Dacă utilizatorul cere expres un anumit ID, folosește-l numai dacă este pregătit.

D. Citește contextul minim necesar, ultima decizie relevantă, rubricile și testele legate de pas.

E. Formulează în 5–10 rânduri obiectivul concret, riscurile și planul de test.

F. Revendică taskul/branchul; implementează exclusiv scopul aprobat.

G. Verifică: tipuri, teste, securitate relevantă, regresii, comportamentul cerut.

H. Actualizează jurnalul, motivele deciziilor, versiunea taskului, dovezile, lecțiile extrase, backlogul și starea.

I. Creează commit/push; propune PR/review unde este cazul, sau marchează BLOCAJ. Nu face merge pe main dacă regulile de review/CI nu sunt îndeplinite.

J. Raportează în română: exact ce a funcționat, ce nu a fost testat, statusul real, commit/PR și ce model să selecteze utilizatorul data viitoare.

K. OPREȘTE COMPLET EXECUȚIA. Nu continua cu etapa următoare fără noul mesaj al utilizatorului.

Dacă un pas este prea mare pentru o sesiune sau quota este aproape de limită, înregistrează un checkpoint autentic și împarte-l în microtaskuri cu aceeași numerotare ierarhică, fără a marca pasul părinte DONE.

20. PRIMA COMANDĂ — ACUM

EXECUTĂ EXCLUSIV PASUL 00 DIN PLANUL DE CONSTRUCȚIE ASOCIAT.

Scop: verifică accesul GitHub, creează un repository NOU pentru RevenuePilot AI (separat de fabrica existentă), inițializează guvernanța minimă, importă în repository această specificație și planul canonic, documentează rezultatul și oprește-te. Dacă accesul nu permite creare remote, explică exact care este blocajul și singura acțiune necesară din telefon. Nu începe implementarea marketingului în această sesiune.

Nu inventa servicii, nu folosi API keys inexistente și nu pretinde că hostul este configurat. În următoarea sesiune, când utilizatorul spune „Continuă cu pasul următor”, citește planul și starea din noul repository și execută exclusiv următoarea etapă eligibilă.

CE ÎNSEAMNĂ SUCCESUL: produs software testat, documentat și construit în repo separat, două modele ce pot continua lucrul unul după altul fără pierderea stării, filtre de calitate calibrabile și învățare din rezultate REALE, capabilități comerciale progresive, autonomie autorizată și costuri sub control. NU înseamnă doar un raport, un număr de agenți sau un site frumos fără flux funcțional.

SFÂRȘITUL META-PROMPTULUI.